

What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm that organizes a program around **objects** instead of only functions.

An object is a real-world entity that contains both:

- **Attributes (Data):** The properties or characteristics of the object.
- **Methods (Behavior):** The actions or operations that the object can perform.

OOP helps us model real-world entities in software, making programs easier to design, understand, maintain, and extend.

Flipkart Example

Imagine you are developing an online shopping platform like **Flipkart**. Every product listed on Flipkart has information such as:

- Product Name
- Brand
- Price
- Category
- Rating
- Stock

A product can also perform actions such as:

- Display product details
- Check stock availability
- Apply a discount
- Calculate the final price

Instead of storing this information in separate variables and writing unrelated functions, OOP groups both the **data (attributes)** and **behavior (methods)** into a single **Product object**.

This makes the program more organized and easier to manage.

Why Do We Need OOP?

For small programs, procedural programming is simple and effective.

However, as applications become larger, managing code using only functions becomes difficult.

Consider an e-commerce application like Flipkart.

It contains:

- Millions of products
- Thousands of sellers
- Millions of customers
- Orders
- Payments
- Delivery partners
- Reviews
- Shopping carts

Managing all these using only variables and functions becomes difficult.

OOP solves this problem by organizing related data and functions into objects.

This results in cleaner, reusable, and scalable code.

Problems with Procedural Programming

In procedural programming:

- Data and functions are separate.
- Code is often repeated.
- Large programs become difficult to maintain.
- Reusing code becomes challenging.
- Real-world entities are difficult to represent.
- Modifying existing features may affect other parts of the program.

OOP Solution

OOP groups related data and functions into a single unit called an object.

For example, instead of creating separate variables and functions for every Flipkart product, we define a single **Product** class and create multiple product objects.

This significantly reduces code duplication and improves maintainability.

Procedural Programming vs Object-Oriented Programming

Procedural Programming Example

Suppose Flipkart sells three products.

Python

```
laptop_name = "HP Laptop"  
laptop_price = 55000  
  
mobile_name = "Samsung Galaxy"  
mobile_price = 28000  
  
headphone_name = "Boat Rockerz"  
headphone_price = 1999
```

Separate functions are required for every product.

Python

```
display_laptop()  
display_mobile()  
display_headphone()
```

As the number of products increases to thousands, the code becomes repetitive and difficult to manage.

Object-Oriented Programming Example

Instead of creating separate variables and functions, we define one Product class.

Python

```
class Product:  
    pass
```

Now we can create multiple product objects.

Python

```
laptop = Product()  
mobile = Product()  
headphones = Product()  
television = Product()
```

Every product follows the same structure, making the application easier to maintain and extend.

Advantages of OOP

1. Modularity

OOP divides a large application into smaller classes.

Each class performs a specific task, making the code easier to understand and maintain.

2. Reusability

A class can be reused multiple times.

For example, the same **Product** class can be used to create laptops, mobiles, televisions, watches, and many other products.

3. Scalability

As Flipkart adds new product categories and features, the existing classes can be extended without rewriting the entire application.

4. Abstraction

Abstraction hides unnecessary implementation details and exposes only the required functionality.

Users only interact with the features they need without knowing the internal implementation.

5. Encapsulation

Encapsulation combines data and methods into a single unit and protects the data from unauthorized access.

This improves data security and prevents accidental modification.

6. Inheritance

Inheritance allows one class to acquire the properties and methods of another class.

This promotes code reuse and reduces duplication.

7. Polymorphism

Polymorphism allows the same method to perform different tasks depending on the object using it.

It makes applications more flexible and easier to extend.

Classes

A **class** is a **blueprint** or **template** used to create objects.

It defines the **attributes (data)** and **methods (behavior)** that every object created from the class will have.

A class itself is only a design. It does not represent a real object until an object is created from it.

Flipkart Example

Suppose you are developing an online shopping platform like **Flipkart**.

Every product listed on Flipkart has:

- Product Name
- Brand
- Price
- Stock
- Category

Every product can also perform actions such as:

- Display product details
- Check stock availability
- Apply discounts

Instead of writing separate variables and functions for every product, we define a **Product** class that acts as a blueprint for all products.

Example

Python

```
class Product:

    platform = "Flipkart"

    def display_product(self):
        print("Displaying Product
Details")

    def check_stock(self):
        print("Stock Available")
```

Here,

- `Product` is the class.
- `platform` is an attribute.
- `display_product()` and `check_stock()` are methods.

At this stage, no actual product exists. The class simply defines what every product should contain.

Objects

An **object** is an **instance of a class**. Objects are the real entities created from a class. A single class can be used to create multiple objects.

Flipkart Example

The **Product** class is the blueprint.

Products such as:

- HP Laptop
- Samsung Galaxy
- Boat Earbuds
- LG Smart TV

are all objects created from the Product class.

Just as one blueprint can be used to build many houses, one class can be used to create many objects.

Creating Objects

Objects are created by calling the class name.

Syntax

```
Python
object_name = ClassName()
```

Example

```
Python
class Product:
    platform = "Flipkart"
```

```
laptop = Product()  
mobile = Product()  
headphones = Product()
```

Here,

- `laptop` is an object.
- `mobile` is another object.
- `headphones` is another object.

All three objects are created from the same Product class.

Creating Multiple Objects

A class can create any number of objects.

```
Python  
class Product:  
    platform = "Flipkart"  
  
product1 = Product()  
product2 = Product()  
product3 = Product()  
product4 = Product()  
product5 = Product()
```

Each object represents a different product, even though they are all created from the same class.

Attributes

Attributes are **variables** that belong to a class or an object.

They store information about an object.

For a Flipkart product, attributes may include:

- Product Name
- Brand

- Price
- Category
- Rating
- Stock

These attributes describe a product.

Example

Python

```
class Product:
    platform = "Flipkart"
    category = "Electronics"
```

Here,

- `platform`
- `category`

are attributes of the Product class.

Accessing Attributes

Attributes are accessed using the **dot (.) operator**.

Syntax

Python

```
object_name.attribute_name
```

or

Python

```
ClassName.attribute_name
```

Accessing Using an Object

Python

```
class Product:
    platform = "Flipkart"
laptop = Product()
print(laptop.platform)
```

Output

```
Python  
Flipkart
```

Accessing Using the Class Name

```
Python  
class Product:  
    platform = "Flipkart"  
print(Product.platform)
```

Output

```
Python  
Flipkart
```

Both approaches access the same class attribute.

Accessing Objects

Once an object is created, it can access the attributes and methods defined inside the class.

```
Python  
class Product:  
    platform = "Flipkart"  
    def display_product(self):  
        print("Displaying Product  
Details")  
laptop = Product()  
print(laptop.platform)  
laptop.display_product()
```

Output

```
Python  
Flipkart  
Displaying Product Details
```

The object `laptop` accesses both the attribute and the method using the dot (.) operator.

Types of Attributes

Attributes are variables that store information about a class or an object.

There are **two types of attributes** in Python:

1. Class Attributes
2. Instance Attributes

1. Class Attributes

A **class attribute** belongs to the class itself.

It is shared by all objects created from that class.

If the value of a class attribute changes, the updated value is reflected in every object unless an object overrides it.

Flipkart Example

Suppose every product on Flipkart belongs to the same shopping platform.

Instead of storing "`Flipkart`" separately for every product, we can store it once as a class attribute.

Python

```
class Product:
    platform = "Flipkart"
```

Here, `platform` is a **class attribute**.

Accessing a Class Attribute

Using the class name:

Python

```
class Product:
    platform = "Flipkart"
print(Product.platform)
```

Output

None

Flipkart

Using an object:

Python

```
class Product:
    platform = "Flipkart"
laptop = Product()
print(laptop.platform)
```

Output

None

Flipkart

Both the class and its objects can access a class attribute.

2. Instance Attributes

An **instance attribute** belongs to a specific object.

Each object has its own copy of an instance attribute.

For example, every Flipkart product has different values for:

- Product Name
- Price
- Brand
- Stock

These values are unique for every product, so they are stored as **instance attributes**.

Note: Instance attributes are usually created inside the **constructor** (`__init__`), which will be discussed in the next chapter.

Methods

A **method** is a function defined inside a class.

Methods describe the behavior or actions that an object can perform.

In a Flipkart application, a product can perform actions such as:

- Display product details
- Check stock availability
- Apply discounts
- Calculate the final price

These actions are represented using methods.

Example

Python

```
class Product:
    def display_product(self):
        print("Displaying Product
Details")
    def check_stock(self):
        print("Stock Available")
```

Here,

- `display_product()` is a method.
- `check_stock()` is another method.

Calling Methods

Methods are called using the **dot (.) operator**.

Python

```
class Product:
    def display_product(self):
        print("Displaying Product
Details")
laptop = Product()
laptop.display_product()
```

Output

None

Displaying Product Details

Types of Methods

Python provides three types of methods.

1. Instance Methods
2. Class Methods
3. Static Methods

1. Instance Methods

An **instance method** works with a specific object.

It is the most commonly used type of method.

An instance method receives **self** as its first parameter.

The **self** parameter represents the current object that calls the method.

Example

```
Python
class Product:
    def display_product(self):
        print("Displaying Product
Details")

laptop = Product()
laptop.display_product()
```

Output

```
None
Displaying Product Details
```

The method is called using the object.

```
Python
laptop.display_product()
```

2. Class Methods

A **class method** works with class-level data.

It is defined using the `@classmethod` decorator.

The first parameter of a class method is `cls`, which refers to the class itself.

Flipkart Example

Suppose Flipkart changes the delivery charge for all products.

Instead of updating every object individually, we update the class attribute.

Python

```
class Product:
    delivery_charge = 40
    @classmethod
    def update_delivery_charge(cls):
        cls.delivery_charge = 60

Product.update_delivery_charge()
print(Product.delivery_charge)
```

Output

None

60

Class methods are generally used to modify or access class attributes.

3. Static Methods

A **static method** does not depend on either the object or the class.

It is defined using the `@staticmethod` decorator.

Static methods are mainly used for utility or helper functions.

Flipkart Example

Suppose we want to check whether a product is eligible for free delivery.

This operation does not require access to any object or class data.

Python

```
class Product:
```

```
@staticmethod
def free_delivery(price):
    return price >= 500

print(Product.free_delivery(800))
print(Product.free_delivery(300))
```

Output

```
None
True
False
```

Accessing Attributes

Attributes store information about a class or an object.

They can be accessed using the **dot (.) operator**.

Syntax

```
Python
object_name.attribute_name
```

or

```
Python
ClassName.attribute_name
```

Accessing a Class Attribute Using an Object

```
Python
class Product:
    platform = "Flipkart"

laptop = Product()
```

```
print(laptop.platform)
```

Output

```
None  
Flipkart
```

The object `laptop` accesses the `platform` attribute using the dot (`.`) operator.

Accessing a Class Attribute Using the Class Name

```
Python  
class Product:  
    platform = "Flipkart"  
  
print(Product.platform)
```

Output

```
None  
Flipkart
```

Both approaches access the same class attribute.

Accessing Methods

Methods are also accessed using the **dot (`.`) operator**.

Accessing an Instance Method

```
Python  
class Product:  
  
    def display_product(self):  
        print("Displaying Product  
Details")
```

```
laptop = Product()

laptop.display_product()
```

Output

```
None
Displaying Product Details
```

Here, the object `laptop` calls the `display_product()` method.

Accessing a Class Method

```
Python
class Product:
    delivery_charge = 40
    @classmethod
    def show_delivery_charge(cls):
        print(cls.delivery_charge)

Product.show_delivery_charge()
```

Output

```
None
40
```

A class method can also be called using an object.

```
Python
product = Product()
product.show_delivery_charge()
```

Output

None

40

Accessing a Static Method

Python

```
class Product:
    @staticmethod
    def free_delivery(price):
        return price >= 500

print(Product.free_delivery(700))
```

Output

None

True

Static methods can also be called using an object.

Python

```
product = Product()
print(product.free_delivery(300))
```

Output

None

False

Flipkart Example

Python

```
class Product:

    platform = "Flipkart"

    def display_product(self):
        print("Welcome to Flipkart")

    @classmethod
    def show_platform(cls):
        print(cls.platform)

    @staticmethod
    def free_delivery(price):
        return price >= 500

laptop = Product()

print(laptop.platform)

laptop.display_product()

Product.show_platform()

print(Product.free_delivery(700))
```

Output

None

```
Flipkart
Welcome to Flipkart
Flipkart
True
```